



Laços e Funções

Aula 04 · Bloco 1 — Fundamentos

Repetir sem repetir código — e o conceito central do curso

Nesta aula

1. `while` — repita **enquanto**
2. `for` — o laço com contador embutido
3. `break` e `continue`
4. **Funções** — parâmetros, `return` e escopo

while — repita enquanto for verdade

```
let contador = 1;

while (contador <= 5) {
  console.log(`Volta número ${contador}`);
  contador++;      // ⚠ sem isto: loop infinito
}
```

⚠ Esqueceu o `contador++` ? O programa nunca para. **Ctrl+C** no terminal interrompe — e vale já saber disso agora.

for — os três slots

```
//  inicialização; condição; incremento  
for (let i = 1; i <= 5; i++) {  
  console.log(`Volta número ${i}`);  
}
```

Os três "slots" organizam, numa linha só, o que no `while` ficava espalhado por três lugares diferentes.

É por isso que o `for` é o laço mais usado quando você **sabe quantas voltas** quer.

Três clássicos

```
for (let i = 10; i >= 1; i--) {      // contagem regressiva
  console.log(i);
}

let soma = 0;                        // somando de 1 a 100
for (let i = 1; i <= 100; i++) soma += i;
console.log(soma);                  // 5050

for (let i = 1; i <= 10; i++) {      // tabuada do 7
  console.log(`7 x ${i} = ${7 * i}`);
}
```

break e continue

```
for (let i = 1; i <= 10; i++) {  
  if (i === 5) break;           // encerra o laço na hora  
  console.log(i);              // 1, 2, 3, 4  
}  
  
for (let i = 1; i <= 5; i++) {  
  if (i === 3) continue;       // pula SÓ esta volta  
  console.log(i);              // 1, 2, 4, 5  
}
```

break sai do laço. **continue** pula para a próxima volta.

Funções

O conceito mais importante do curso.

Um **bloco de código com nome**, que você define uma vez e executa quantas vezes quiser.

Tudo daqui em diante é feito de funções.

Definir e chamar

```
// definição
function saudacao() {
  console.log("Olá! Bem-vindo(a).");
}

// chamada
saudacao();
saudacao();    // reutilização – escreveu uma vez, usa sempre
```


Parâmetros e argumentos

```
function saudacao(nome) {           // nome é o PARÂMETRO
  console.log(`Olá, ${nome}!`);
}

saudacao("Maria");                  // "Maria" é o ARGUMENTO
saudacao("João");
```

O **parâmetro** é o nome que a função usa por dentro. O **argumento** é o valor que você entrega na hora de chamar.

return — devolvendo um resultado

```
function somar(a, b) {  
  return a + b;           // devolve para quem chamou  
}  
  
const resultado = somar(3, 4);      // 7  
console.log(somar(10, somar(1, 2))); // 13 – funções compõem!
```

⚠ O **return** **encerra a função** ali mesmo. Código depois dele nunca executa. E função sem **return** devolve **undefined**.

A diferença que confunde todo mundo

console.log *mostra* um valor na tela.

É para o **humano** ver.

return *devolve* um valor para o programa.

É para o **código** usar.

```
function dobro(n) { return n * 2; }  
console.log(dobro(dobro(5))); // 20 - só funciona porque RETORNA
```

Combinando tudo

```
function classificar(media) {  
  if (media >= 7) return "Aprovado";  
  if (media >= 4) return "Recuperação";  
  return "Reprovado";  
}  
  
for (let nota = 0; nota <= 10; nota += 2) {  
  console.log(`${nota}: ${classificar(nota)}`);  
}
```

Escopo — onde as variáveis existem

```
function teste() {  
  const local = "só existo dentro da função";  
  console.log(local);    // ✓  
}  
  
teste();  
console.log(local);      // ✗ ReferenceError
```

O que nasce dentro de uma função **só existe ali**. Isso é bom: impede que pedaços do programa se atrapalhem.

💡 Ponte com o Java

Função em JavaScript $\approx$ método `static` em Java.

```
function somar(a, b)    →    static int somar(int a, int b)
```

Mesma ideia. A diferença é que o Java exige os tipos escritos.

Exercícios da aula

Na pasta `aula-04/` :

1. `ex01.js` — tabuada completa de um número;
2. `ex02.js` — soma dos pares de 1 a 1000 (esperado: 250500);
3. `ex03.js` — `celsiusParaFahrenheit(c)` + tabela de 0 a 40°C;
4. `ex04.js` — `ehPrimo(n)` e os primos até 50;
5. `ex05.js` — `fatorial(n)` . E `fatorial(0)` , quanto dá?
6. **Desafio** 🌶️ `ex06.js` — **FizzBuzz**, o clássico de entrevista.

Fim do Bloco 1

Você já sabe o essencial de lógica em JavaScript:
variáveis, decisões, repetição e funções.

Bloco 2 — Estruturas de Dados

Arrays, objetos e os métodos que
transformam coleções inteiras de uma vez.